Ministry of Education, Culture and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatic Control

ANDREI ELCISCEV
Group FAF-243

LABORATORY WORK No. 7

Three Small Programs: Processes, Redirection, a Pipe

Operating Systems

Checked by: Reiţman Patricia

Chişinău, 2026

Contents

1 Objective **2**

2 Working Environment **2**

3 Part 1: Process Creation with `fork()`, `exec()`, and `wait()` **2**

3.1 Implementation and execution 2

3.2 Changing the executed program 3

3.3 Observations and answers 3

4 Part 2: Observing a Zombie Process **4**

4.1 Implementation and execution 4

4.2 Observations and answers 5

5 Part 3: Output Redirection **5**

5.1 Implementation and execution 5

5.2 Observations and answers 6

6 Part 4: Creating a Pipe **6**

6.1 Implementation and execution 6

6.2 Breaking the pipeline 7

6.3 Observations and answers 7

7 Conclusion **7**

1 Objective

The objective of this laboratory work is to examine process creation and execution in Linux, observe zombie processes, implement output redirection, and build a pipeline between two programs using system calls. The experiments demonstrate the roles of `fork()`, `exec()`, `waitpid()`, file descriptors, `dup()`, and `pipe()`.

2 Working Environment

The experiments were performed in a Linux environment. The source files were compiled using GCC with warning flags enabled.

```
mkdir -p ~/os-lab7
cd ~/os-lab7
gcc --version | head -1
```

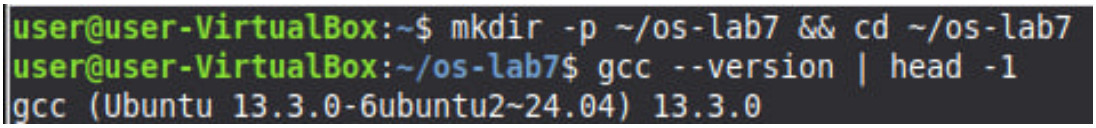


Figure 1: Working directory and GCC version

3 Part 1: Process Creation with `fork()`, `exec()`, and `wait()`

3.1 Implementation and execution

The program `run.c` demonstrates the basic process lifecycle used by a shell. The parent process prints its PID and calls `fork()`. The child prints its PID and its parent's PID, then calls `execlp()` to replace its process image with the `ls -l` program. The parent calls `waitpid()` and prints the child's exit status.

The program was compiled and executed with:

```
gcc -Wall -Wextra -o run run.c
./run
```

```
user@user-VirtualBox:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 1980
child:  my PID is 1981, my parent is 1980
total 24
-rwxrwxr-x 1 user user 16400 Oct 19 14:06 run
-rw-rw-r-- 1 user user   937 Oct 19 14:03 run.c
parent: child 1981 ended with status 0
```

Figure 2: Part 1: output of run executing ls -l

3.2 Changing the executed program

The `execlp()` call was changed to execute the `date` command:

```
execlp("date", "date", (char *)NULL);
```

The program was recompiled and executed to observe how the child process runs a different program.

```
user@user-VirtualBox:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 2806
child:  my PID is 2807, my parent is 2806
Mon Oct 19 02:13:51 PM EEST 2026
parent: child 2807 ended with status 0
```

Figure 3: Part 1: output after changing execlp to date

Next, the program was changed to attempt to execute a nonexistent command:

```
execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

If `execlp()` fails, execution continues to `perror("exec")` and the child exits with status 127.

```
user@user-VirtualBox:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 2824
child:  my PID is 2825, my parent is 2824
exec: No such file or directory
parent: child 2825 ended with status 127
```

Figure 4: Part 1: output when execlp attempts to run nosuchprog

3.3 Observations and answers

1. **Why is `perror("exec")` printed only with `nosuchprog`?**
   A successful `execlp()` replaces the child process image and does not return. Therefore, the next line is reached only when execution fails, as it does for `nosuchprog`.

2. **Which number does the child print as "my parent", and whose PID is it?**
   The number is the PID printed earlier by the parent process. It is returned to the child by `getppid()` and identifies the child's parent process.

3. **What status does the parent see with `ls`, and what status with `nosuchprog`?** With `ls`, the parent normally observes exit status 0, indicating success. With `nosuchprog`, the child calls `exit(127)` after `execlp()` fails, so the parent observes status 127.

4 Part 2: Observing a Zombie Process

4.1 Implementation and execution

A zombie process is a child process that has terminated but whose termination status has not yet been collected by its parent. To demonstrate this, the original `run.c` program was copied to `zombie.c`, and `sleep(20);` was added immediately before `waitpid()`.

The relevant code is:

```
int status;
sleep(20);
waitpid(pid, &status, 0);
```

The program was compiled and run in the background. The process table was inspected while the parent was sleeping:

```
gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2
ps -o pid,ppid,stat,cmd
```

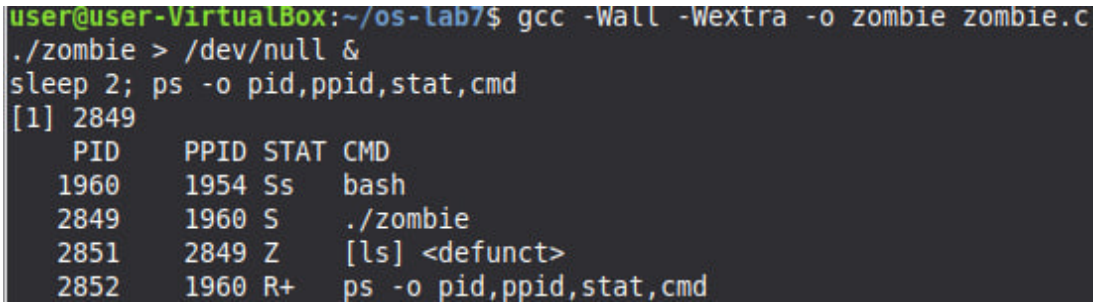


Figure 5: Part 2: process table showing the zombie child

After waiting approximately 20 seconds, the process table was inspected again:

```
ps -o pid,ppid,stat,cmd
```

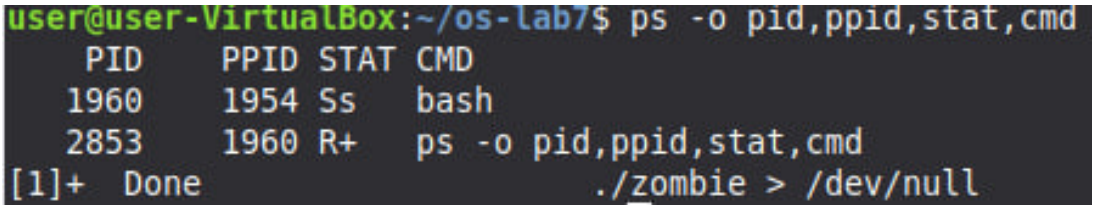


Figure 6: Part 2: process table after the parent has waited

4.2 Observations and answers

1. **Which letter does `ls` have in the `STAT` column, and what does it mean?**
   The child `ls` process is marked with `Z`. This means it is a zombie: it has terminated, but its parent has not yet collected its exit status.

2. **After 20 seconds, is the `[ls] <defunct>` line still there? Which line made it disappear?**
   Once the parent resumes and successfully executes `waitpid(pid, &status, 0)`, the child's termination status is collected and the zombie entry is removed. Therefore, the `waitpid()` line makes the zombie disappear, assuming the process has already terminated.

5 Part 3: Output Redirection

5.1 Implementation and execution

The program `redirect.c` implements the behavior of the shell command `ls -l > out.txt`. The child closes file descriptor 1, which represents standard output, and opens `out.txt`. Since descriptor 1 is the smallest available descriptor, `open()` assigns it to the new file. When `ls` runs, its standard output therefore points to `out.txt`.

The program was compiled and executed with:

```
gcc -Wall -Wextra -o redirect redirect.c
./redirect
head -3 out.txt
```

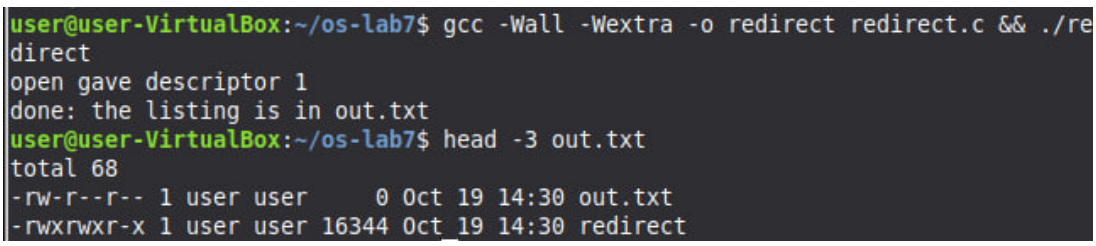


Figure 7: Part 3: program output and first lines of out.txt

5.2 Observations and answers

1. **Which number did `open()` give, and why exactly that one?**
   It gave descriptor 1 because the child first closed descriptor 1. The operating system assigns the lowest-numbered available file descriptor, and descriptor 1 was the lowest available one.

2. **Why does the listing not appear on the screen even though `ls` was not changed?**
   Programs conventionally write standard output to file descriptor 1. Because descriptor 1 now refers to `out.txt`, the listing is written to that file instead of the terminal.

3. **To redirect errors as in `2> err.txt`, which descriptor would you close before opening the file?**
   Close file descriptor 2, which represents standard error, before calling `open()`. The newly opened file should then receive descriptor 2, provided it is the lowest-numbered available descriptor.

6 Part 4: Creating a Pipe

6.1 Implementation and execution

The program `pipeline.c` implements the equivalent of the shell pipeline:

```
ls /etc | wc -l
```

The `pipe(p)` call creates a pair of file descriptors: `p[0]` is the reading end and `p[1]` is the writing end. The first child redirects its standard output to the pipe's writing end and executes `ls`. The second child redirects its standard input to the reading end and executes `wc -l`. The parent closes both ends and waits for both children.

The program and shell command were run for comparison:

```
gcc -Wall -Wextra -o pipeline pipeline.c
./pipeline
ls /etc | wc -l
```

```
user@user-VirtualBox:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pi
peline
253
user@user-VirtualBox:~/os-lab7$ ls /etc | wc -l
253
```

Figure 8: Part 4: comparing pipeline output with the shell pipeline

6.2 Breaking the pipeline

The parent's call to `close(p[1])` was commented out, leaving the parent's writing end open. The program was recompiled and executed again. The process was stopped with Ctrl-C after observing that it did not finish.

```
user@user-VirtualBox:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pi
peline
^C
user@user-VirtualBox:~/os-lab7$
```

Figure 9: Part 4: demonstration of the pipeline hanging

6.3 Observations and answers

1. **Did `./pipeline` and `ls /etc | wc -l` give the same number?**
   Yes. When the pipeline is implemented correctly, both commands should count the same number of entries in `/etc`, assuming the directory contents do not change between executions.

2. **After the change, which child never ends, `ls` or `wc`?**
   The `wc` child does not finish because it continues waiting for end-of-file on its standard input.

3. **Why, in one sentence, is `wc` waiting?**
   The parent still has the pipe's writing end open, so `wc` does not receive end-of-file even after `ls` exits and closes its own writing descriptor.

7 Conclusion

During this laboratory work, process creation and program replacement were examined using `fork()` and `exec()`, while `waitpid()` was used to collect a child process's termination status. A zombie process was observed when the parent delayed waiting for its child. Output redirection was implemented by manipulating file descriptors, and a pipeline was constructed using `pipe()` and `dup()`. The final experiment demonstrated that all unused pipe ends must be closed; otherwise, a reading process may wait indefinitely for end-of-file.